

DSA Assignment AVL Tree Problem

Name :
AHDHARSH A

Roll No: 25BAD003

Question 1

Insert into an AVL Tree while Counting and Removing Duplicate Keys

//25BAD003 AHDHARSH A

//Module 6_AVL Trees_Q1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {    int key;  
    struct Node *left;    struct  
    Node *right;    int height;  
};
```

```
int max(int a, int b) {    return (a > b) ?  
a : b;  
}
```

```
int height(struct Node *n) {    if (n ==  
NULL) return 0;    return n->height;  
}
```

```
struct Node* newNode(int key) {  
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));    node->key = key;  
    node->left = NULL;    node->right = NULL;    node->height = 1;    return node;  
}
```

```
struct Node* rightRotate(struct Node *y) {    struct  
Node *x = y->left;    struct Node *T2 = x->right;
```

```
    x->right = y;    y->left = T2;
```

```
    y->height = max(height(y->left), height(y->right)) + 1;    x->height = max(height(x->left),  
height(x->right)) + 1;
```

```
    return x;  
}
```

```
struct Node* leftRotate(struct Node *x) {
```

```

    struct Node *y = x->right;    struct Node *T2
= y->left;

    y->left = x;    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;    y->height = max(height(y->left),
height(y->right)) + 1;

    return y;
}

int getBalance(struct Node *n) {    if (n ==
NULL) return 0;
    return height(n->left) - height(n->right);
}

struct Node* insert(struct Node* node, int key, int *isDuplicate) {    if (node == NULL) {
return newNode(key);
}

    if (key < node->key) {
        node->left = insert(node->left, key, isDuplicate);
    } else if (key > node->key) {
        node->right = insert(node->right, key, isDuplicate);    } else {
        *isDuplicate = 1;        return node;
    }

    node->height = 1 + max(height(node->left), height(node->right));    int balance =
getBalance(node);

    if (balance > 1 && key < node->left->key)        return
rightRotate(node);

    if (balance < -1 && key > node->right->key)        return
leftRotate(node);

    if (balance > 1 && key > node->left->key) {        node->left =
leftRotate(node->left);        return rightRotate(node);
    }

    if (balance < -1 && key < node->right->key) {        node->right =
rightRotate(node->right);        return leftRotate(node);
    }

    return node;
}

struct Node* minValueNode(struct Node* node) {    struct
Node* current = node;    while (current->left != NULL)
current = current->left;    return current;
}

struct Node* deleteNode(struct Node* root, int key) {

```

```

if (root == NULL) return root;

if (key < root->key)
    root->left = deleteNode(root->left, key); else if (key > root->key)
    root->right = deleteNode(root->right, key); else {
    if ((root->left == NULL) || (root->right == NULL)) {        struct Node *temp = root-
>left ? root->left : root->right;        if (temp == NULL) {        temp = root;
root = NULL;
    } else {
        *root = *temp;
    }
    free(temp);        } else {
    struct Node* temp = minValueNode(root->right);        root->key = temp-
>key;        root->right = deleteNode(root->right, temp->key);        }
    }

if (root == NULL) return root;

root->height = 1 + max(height(root->left), height(root->right));    int balance =
getBalance(root);

if (balance > 1 && getBalance(root->left) >= 0)        return
rightRotate(root);

if (balance > 1 && getBalance(root->left) < 0) {        root->left =
leftRotate(root->left);        return rightRotate(root);
}

if (balance < -1 && getBalance(root->right) <= 0)        return leftRotate(root);

if (balance < -1 && getBalance(root->right) > 0) {        root->right =
rightRotate(root->right);        return leftRotate(root);
}

return root;
}

void inorder(struct Node *root) {    if (root !=
NULL) {        inorder(root->left);
printf("%d ", root->key);        inorder(root-
>right);
    }
}

int main() {    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct Node *root = NULL;    int
dupCount = 0;

```

```

    for (int i = 0; i < n; i++) {        int val;
scanf("%d", &val);        int isDup = 0;
    root = insert(root, val, &isDup);    if (isDup) {
dupCount++;
    }
    }

    printf("AVL Tree after removing duplicates: ");    inorder(root);
printf("\n");

    printf("Total number of duplicates: %d\n", dupCount);

    int m;
    if (scanf("%d", &m) == 1) {        for (int i = 0; i < m;
i++) {            int delVal;        scanf("%d", &delVal);
root = deleteNode(root, delVal);
        }
    }

    printf("AVL Tree after deletions: ");    inorder(root);
printf("\n");

    return 0;
}

```

Question 2

AVL Tree with Associated String Data and Deletion of Minimum Key

```

//25BAD026 GOWTHAM D M
//Module 6_AVL Trees_Q2

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

struct Node {    int key;
char code[32];    struct Node
*left;    struct Node *right;
int height;
};

```

```

int max(int a, int b) {    return (a > b) ?
a : b;
}

```

```

int height(struct Node *n) {    if (n ==
NULL) return 0;

```

```

    return n->height;
}

```

```

struct Node* newNode(int key, char* code) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));  node->key = key;
    strcpy(node->code, code);  node->left = NULL;  node->right = NULL;  node-
    >height = 1;
    return node;
}

```

```

struct Node* rightRotate(struct Node *y) {  struct
Node *x = y->left;  struct Node *T2 = x->right;

```

```

    x->right = y;  y->left = T2;

```

```

    y->height = max(height(y->left), height(y->right)) + 1;  x->height = max(height(x->left),
height(x->right)) + 1;

```

```

    return x;
}

```

```

struct Node* leftRotate(struct Node *x) {  struct Node
*y = x->right;  struct Node *T2 = y->left;

```

```

    y->left = x;  x->right = T2;

```

```

    x->height = max(height(x->left), height(x->right)) + 1;  y->height = max(height(y->left),
height(y->right)) + 1;

```

```

    return y;
}

```

```

int getBalance(struct Node *n) {  if (n ==
NULL) return 0;
    return height(n->left) - height(n->right);
}

```

```

struct Node* insert(struct Node* node, int key, char* code) {  if (node == NULL)
{  return newNode(key, code);  }

```

```

    if (key < node->key) {  node->left = insert(node->left, key,
code);
    } else if (key > node->key) {  node->right = insert(node->right,
key, code);
    } else {
        return node;
    }
}

```

```

    node->height = 1 + max(height(node->left), height(node->right));  int balance =
getBalance(node);

```

```

    if (balance > 1 && key < node->left->key)

```

```

    return rightRotate(node);

    if (balance < -1 && key > node->right->key)    return
leftRotate(node);

    if (balance > 1 && key > node->left->key) {    node->left =
leftRotate(node->left);    return rightRotate(node);
    }

    if (balance < -1 && key < node->right->key) {    node->right =
rightRotate(node->right);    return leftRotate(node);
    }

    return node;
}

struct Node* minValueNode(struct Node* node) {    struct
Node* current = node;    while (current->left != NULL)
current = current->left;    return current;
}

struct Node* deleteNode(struct Node* root, int key) {    if (root == NULL)
return root;

    if (key < root->key)
        root->left = deleteNode(root->left, key);    else if (key > root-
>key)    root->right = deleteNode(root->right, key);    else {
        if ((root->left == NULL) || (root->right == NULL)) {    struct Node *temp = root-
>left ? root->left : root->right;    if (temp == NULL) {    temp = root;
root = NULL;
        } else {
            *root = *temp;
        }
        free(temp);    } else {    struct Node* temp = minValueNode(root-
>right);    root->key = temp->key;    strcpy(root->code, temp->code);
root->right = deleteNode(root->right, temp->key);    }
    }

    if (root == NULL) return root;

    root->height = 1 + max(height(root->left), height(root->right));    int balance =
getBalance(root);

    if (balance > 1 && getBalance(root->left) >= 0)    return
rightRotate(root);

    if (balance > 1 && getBalance(root->left) < 0) {    root->left =
leftRotate(root->left);    return rightRotate(root);
    }

```

```

    }

    if (balance < -1 && getBalance(root->right) <= 0)    return leftRotate(root);

    if (balance < -1 && getBalance(root->right) > 0) {    root->right =
rightRotate(root->right);    return leftRotate(root);
    }

    return root;
}

void inorder(struct Node *root) {    if (root !=
NULL) {        inorder(root->left);
        printf("%d %s\n", root->key, root->code);    inorder(root->right);
    }
}

int main() {    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct Node *root = NULL;    for (int i = 0;
i < n; i++) {        int k;        char c[32];
scanf("%d %s", &k, c);        root =
insert(root, k, c);
    }

    printf("In-order traversal before deletion:\n");    inorder(root);

    if (root != NULL) {        struct Node *minNode =
minValueNode(root);        root = deleteNode(root, minNode->key);
    }

    printf("In-order traversal after deletion:\n");    inorder(root);

    return 0;
}

```

Question 3

Search for a Key in an AVL Tree

//25BAD026 GOWTHAM D M

//Module 6_AVL Trees_Q3

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {

```

```

    int key;    struct Node *left;
struct Node *right;    int
height;
};

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getHeight(struct Node *n) {    if (n ==
NULL) return 0;    return n->height;
}

struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));    node->key = key;
node->left = NULL;    node->right = NULL;    node->height = 1;
    return node;
}

struct Node* rightRotate(struct Node *y) {    struct
Node *x = y->left;    struct Node *T2 = x->right;

    x->right = y;    y->left = T2;

    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;    x->height = max(getHeight(x-
>left), getHeight(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node *x) {    struct Node
*y = x->right;    struct Node *T2 = y->left;

    y->left = x;    x->right = T2;

    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;    y->height = max(getHeight(y-
>left), getHeight(y->right)) + 1;

    return y;
}

int getBalance(struct Node *n) {    if (n == NULL) return 0;    return
getHeight(n->left) - getHeight(n->right);
}

struct Node* insert(struct Node* node, int key) {    if (node ==
NULL)        return createNode(key);

    if (key < node->key)        node->left = insert(node->left,
key);

```

```

else if (key > node->key)
    node->right = insert(node->right, key); else
    return node;

node->height = 1 + max(getHeight(node->left), getHeight(node->right));

int balance = getBalance(node);

// Left Left Case
if (balance > 1 && key < node->left->key) return
rightRotate(node);

// Right Right Case
if (balance < -1 && key > node->right->key) return
leftRotate(node);

// Left Right Case
if (balance > 1 && key > node->left->key) { node->left =
leftRotate(node->left); return rightRotate(node);
}

// Right Left Case if (balance < -1 && key < node->right-
>key) { node->right = rightRotate(node->right); return
leftRotate(node);
}

return node;
}

int search(struct Node* root, int key) { if (root ==
NULL) return 0; if (root->key == key)
return 1; if (key < root->key) return
search(root->left, key); return search(root->right,
key);
}

int main() { struct Node *root =
NULL; int val;

while (scanf("%d", &val) == 1 && val != -1) { root = insert(root,
val);
}

int k; if (scanf("%d", &k) == 1) { if (search(root, k)) {
printf("Key %d found in the AVL tree.\n", k);
} else { printf("Key %d not found in the AVL tree.\n", k); }
}

return 0;
}

```

Question 4

Find the Height of an AVL Tree

////25BAD026 GOWTHAM D M
//Module 6_AVL Trees_Q4

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {    int key;
struct Node *left; struct
Node *right;    int height;
};
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int getHeight(struct Node *n) {    if (n ==
NULL)        return 0;    return n->height;
}
```

```
struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));    node->key = key;
node->left = NULL;    node->right = NULL;    node->height = 1;    return node;
}
```

```
struct Node* rightRotate(struct Node *y) {    struct
Node *x = y->left;    struct Node *T2 = x->right;
```

```
    x->right = y;    y->left = T2;
```

```
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;    x->height = max(getHeight(x-
>left), getHeight(x->right)) + 1;
```

```
    return x;
}
```

```
struct Node* leftRotate(struct Node *x) {    struct Node
*y = x->right;    struct Node *T2 = y->left;
```

```
    y->left = x;    x->right = T2;
```

```
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
```

```

y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

return y;
}

int getBalance(struct Node *n) {    if (n ==
NULL)        return 0;
    return getHeight(n->left) - getHeight(n->right);
}

struct Node* insert(struct Node* node, int key) {    if (node ==
NULL)        return createNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);    else if (key > node-
>key)
        node->right = insert(node->right, key);    else
        return node;

    node->height = 1 + max(getHeight(node->left), getHeight(node->right));

    int balance = getBalance(node);

    // Left Left Case
    if (balance > 1 && key < node->left->key)        return
rightRotate(node);

    // Right Right Case
    if (balance < -1 && key > node->right->key)        return
leftRotate(node);

    // Left Right Case    if (balance > 1 && key > node->left-
>key) {        node->left = leftRotate(node->left);        return
rightRotate(node);
    }

    // Right Left Case    if (balance < -1 && key < node->right-
>key) {        node->right = rightRotate(node->right);        return
leftRotate(node);
    }

    return node;
}

int main() {    struct Node *root =
NULL;    int val;

    while (scanf("%d", &val) == 1 && val != -1) {        root = insert(root,
val);
    }

    printf("Height of the AVL tree: %d\n", getHeight(root));

    return 0;
}

```

```
}
```

Question 5

Preorder Traversal of an AVL Tree Before and After Deletion

```
////25BAD026 GOWTHAM D M
```

```
//Module 6_AVL Trees_Q5
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {    int key;
struct Node *left; struct
Node *right;    int height;
};
```

```
int max(int a, int b) {    return (a > b) ?
a : b;
}
```

```
int getHeight(struct Node *n) {    if (n ==
NULL) return 0;    return n->height;
}
```

```
struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));    node->key = key;
node->left = NULL;    node->right = NULL;    node->height = 1;    return node;
}
```

```
struct Node* rightRotate(struct Node *y) {    struct
Node *x = y->left;    struct Node *T2 = x->right;
```

```
    x->right = y;    y->left = T2;
```

```
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;    x->height = max(getHeight(x-
>left), getHeight(x->right)) + 1;
```

```
    return x;
}
```

```
struct Node* leftRotate(struct Node *x) {    struct Node
*y = x->right;    struct Node *T2 = y->left;
```

```
    y->left = x;    x->right = T2;
```

```
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
```

```

y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

return y;
}

int getBalance(struct Node *n) { if (n ==
NULL) return 0;
return getHeight(n->left) - getHeight(n->right);
}

struct Node* insert(struct Node* node, int key) { if (node ==
NULL) return createNode(key);

if (key < node->key)
node->left = insert(node->left, key); else if (key > node-
>key)
node->right = insert(node->right, key); else
return node;

node->height = 1 + max(getHeight(node->left), getHeight(node->right));

int balance = getBalance(node);

// Left Left
if (balance > 1 && key < node->left->key) return
rightRotate(node);

// Right Right if (balance < -1 && key > node->right-
>key) return leftRotate(node);

// Left Right
if (balance > 1 && key > node->left->key) { node->left =
leftRotate(node->left); return rightRotate(node);
}

// Right Left if (balance < -1 && key < node->right->key) {
node->right = rightRotate(node->right); return
leftRotate(node);
}

return node;
}

struct Node* minValueNode(struct Node* node) { struct
Node* current = node; while (current->left != NULL)
current = current->left; return current;
}

struct Node* deleteNode(struct Node* root, int key) { if (root ==
NULL) return root;

if (key < root->key) root->left = deleteNode(root->left,
key);

```

```

else if (key > root->key)
    root->right = deleteNode(root->right, key); else {
    if ((root->left == NULL) || (root->right == NULL)) { struct Node *temp = root-
>left ? root->left : root->right;

        if (temp == NULL) { temp
= root; root = NULL;
        } else {
            *root = *temp;
        }
        free(temp); } else {
        struct Node* temp = minValueNode(root->right); root->key = temp->key;
        root->right = deleteNode(root->right, temp->key); }
    }

    if (root == NULL) return
root;

    root->height = 1 + max(getHeight(root->left), getHeight(root->right));

    int balance = getBalance(root);

    // Left Left
    if (balance > 1 && getBalance(root->left) >= 0) return
rightRotate(root);

    // Left Right if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left); return rightRotate(root);
    }

    // Right Right
    if (balance < -1 && getBalance(root->right) <= 0) return leftRotate(root);

    // Right Left
    if (balance < -1 && getBalance(root->right) > 0) { root->right =
rightRotate(root->right); return leftRotate(root);
    }

    return root;
}

void printPreorder(struct Node *root) { if (root !=
NULL) { printf("%d ", root->key);
printPreorder(root->left); printPreorder(root-
>right); }
}

int main() { int n; if (scanf("%d", &n) != 1)
return 0;

```

```

    struct Node *root = NULL;    for (int i =
0; i < n; i++) {        int val;        scanf("%d",
&val);        root = insert(root, val);
    }

    int delVal;    scanf("%d",
&delVal);

    printf("Preorder of AVL tree\n");
printPreorder(root);    printf("\n");

    root = deleteNode(root, delVal);

    printf("Preorder after deletion of the element\n");    printPreorder(root);
printf("\n");

    return 0;
}

```

Question 6

Count Elements Greater than a Given Value in an AVL Tree

////25BAD026 GOWTHAM D M

//Module 6_AVL Trees_Q6

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {    int key;
struct Node *left;    struct
Node *right;    int height;
int count;
};

```

```

int max(int a, int b) {    return (a > b) ?
a : b;
}

```

```

int getHeight(struct Node *n) {    if (n ==
NULL) return 0;    return n->height;
}

```

```

struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));    node->key = key;
node->left = NULL;    node->right = NULL;    node->height = 1;    node->count = 1;
}

```

```

    return node;
}

```

```

struct Node* rightRotate(struct Node *y) {    struct
Node *x = y->left;    struct Node *T2 = x->right;

```

```

    x->right = y;    y->left = T2;

```

```

    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;    x->height = max(getHeight(x-
>left), getHeight(x->right)) + 1;

```

```

    return x;
}

```

```

struct Node* leftRotate(struct Node *x) {    struct Node
*y = x->right;    struct Node *T2 = y->left;

```

```

    y->left = x;    x->right = T2;

```

```

    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;    y->height = max(getHeight(y-
>left), getHeight(y->right)) + 1;

```

```

    return y;
}

```

```

int getBalance(struct Node *n) {    if (n ==
NULL) return 0;
    return getHeight(n->left) - getHeight(n->right);
}

```

```

struct Node* insert(struct Node* node, int key) {    if (node ==
NULL)    return createNode(key);

```

```

    if (key < node->key)
        node->left = insert(node->left, key);    else if (key > node-
>key)    node->right = insert(node->right, key);    else {
        node->count++;    return node;
    }
}

```

```

    node->height = 1 + max(getHeight(node->left), getHeight(node->right));

```

```

    int balance = getBalance(node);

```

```

    // Left Left Case    if (balance > 1 && key < node->left-
>key)    return rightRotate(node);

```

```

    // Right Right Case    if (balance < -1 && key > node->right-
>key)    return leftRotate(node);

```

```

    // Left Right Case

```

```

    if (balance > 1 && key > node->left->key) {        node->left =
leftRotate(node->left);        return rightRotate(node);
    }

    // Right Left Case
    if (balance < -1 && key < node->right->key) {        node->right =
rightRotate(node->right);        return leftRotate(node);
    }

    return node;
}

int countGreater(struct Node* root, int x) {    if (root ==
NULL)        return 0;

    if (root->key > x) {
        return root->count + countGreater(root->left, x) + countGreater(root->right, x);
    } else {        return countGreater(root->right, x);
    }
}

int main() {    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct Node *root = NULL;    for (int i =
0; i < n; i++) {        int val;        scanf("%d",
&val);        root = insert(root, val);
    }

    int x;    scanf("%d", &x);

    int count = countGreater(root, x);
    printf("Number of elements greater than %d are %d\n", x, count);

    return 0;
}

```

Repository Link

<https://github.com/ahdharsh>

